

# 13. 反射、枚举以及lambda表达式

## 【本节目标】

- 1. 掌握反射
- 2. 掌握枚举
- 3. 掌握lambda表达式使用

## 1. 反射

### 1.1 定义

Java的反射（reflection）机制是在运行时检查、访问和修改类、接口、字段和方法的机制；这种动态获取信息以及动态调用对象方法的功能称为java语言的反射（reflection）机制。

### 1.2 用途(了解)

- 1. 框架开发
- 2. 注解处理
- 3. 动态代理
- 4. 配置文件解析
- 5. 等等

### 1.3 反射相关的类（重要）

| 类名           | 用途                         |
|--------------|----------------------------|
| Class类       | 代表类的实体，在运行的Java应用程序中表示类和接口 |
| Field类       | 代表类的成员变量/类的属性              |
| Method类      | 代表类的方法                     |
| Constructor类 | 代表类的构造方法                   |

#### 1.3.1 Class类(反射机制的起源)

Class帮助文档代表类的实体，在运行的Java应用程序中表示类和接口。

Java文件被编译后，生成了.class文件，JVM此时就要去解读.class文件 ,被编译后的Java文件.class也被JVM解析为一个对象，这个对象就是 `java.lang.Class` .这样当程序在运行时，每个java文件就最终变成了Class类对象的一个实例。我们通过Java的反射机制应用到这个实例，就可以去获得甚至去添加改变这个类的属性和动作，使得这个类成为一个动态的类。

1.3.1.1 Class类中的相关方法(方法的使用方法在后边的示例当中)

- (重要)常用获得类相关的方法

| 方法                                     | 用途                               |
|----------------------------------------|----------------------------------|
| <code>getClassLoader()</code>          | 获得类的加载器                          |
| <code>getDeclaredClasses()</code>      | 返回一个数组，数组中包含该类中所有类和接口类的对象(包括私有的) |
| <code>forName(String className)</code> | 根据类名返回类的对象                       |
| <code>newInstance()</code>             | 创建类的实例                           |
| <code>getName()</code>                 | 获得类的完整路径名字                       |

- (重要)常用获得类中属性相关的方法(以下方法返回值为Field相关)

| 方法                                         | 用途          |
|--------------------------------------------|-------------|
| <code>getField(String name)</code>         | 获得某个公有的属性对象 |
| <code>getFields()</code>                   | 获得所有公有的属性对象 |
| <code>getDeclaredField(String name)</code> | 获得某个属性对象    |
| <code>getDeclaredFields()</code>           | 获得所有属性对象    |

- (了解)获得类中注解相关的方法

| 方法                                                                 | 用途                  |
|--------------------------------------------------------------------|---------------------|
| <code>getAnnotation(Class&lt;A&gt; annotationClass)</code>         | 返回该类中与参数类型匹配的公有注解对象 |
| <code>getAnnotations()</code>                                      | 返回该类所有的公有注解对象       |
| <code>getDeclaredAnnotation(Class&lt;A&gt; annotationClass)</code> | 返回该类中与参数类型匹配的所有注解对象 |
| <code>getDeclaredAnnotations()</code>                              | 返回该类所有的注解对象         |

- (重要)获得类中构造器相关的方法（以下方法返回值为Constructor相关）

| 方法                                                 | 用途                  |
|----------------------------------------------------|---------------------|
| getConstructor(Class...<?> parameterTypes)         | 获得该类中与参数类型匹配的公有构造方法 |
| getConstructors()                                  | 获得该类的所有公有构造方法       |
| getDeclaredConstructor(Class...<?> parameterTypes) | 获得该类中与参数类型匹配的构造方法   |
| getDeclaredConstructors()                          | 获得该类所有构造方法          |

- (重要)获得类中方法相关的方法（以下方法返回值为[Method](#)相关）

| 方法                                                         | 用途          |
|------------------------------------------------------------|-------------|
| getMethod(String name, Class...<?> parameterTypes)         | 获得该类某个公有的方法 |
| getMethods()                                               | 获得该类所有公有的方法 |
| getDeclaredMethod(String name, Class...<?> parameterTypes) | 获得该类某个方法    |
| getDeclaredMethods()                                       | 获得该类所有方法    |

## 1.3.2 反射示例

### 1.3.2.1 获得Class对象的三种方式

在反射之前，我们需要做的第一步就是先拿到当前需要反射的类的Class对象，然后通过Class对象的核心方法，达到反射的目的，即：在**运行**状态中，对于任意一个类，都能够知道这个类的**所有属性和方法**；对于任意一个对象，都能够调用它的任意方法和属性，既然能拿到那么，我们就可以修改部分类型信息。

**第一种**，使用 `Class.forName("类的全路径名");` 静态方法。

前提：已明确类的全路径名。

**第二种**，使用 `.class` 方法。

说明：仅适合在编译前就已经明确要操作的 Class

**第三种**，使用类对象的 `getClass()` 方法

```

1 class Student {
2     //私有属性name
3     private String name = "bit";
4     //公有属性age
5     public int age = 18;

```

```
6 //不带参数的构造方法
7 public Student(){
8     System.out.println("Student()");
9 }
10
11 private Student(String name,int age) {
12     this.name = name;
13     this.age = age;
14     System.out.println("Student(String,name)");
15 }
16
17 private void eat(){
18     System.out.println("i am eat");
19 }
20
21 public void sleep(){
22     System.out.println("i am pig");
23 }
24
25 private void function(String str) {
26     System.out.println(str);
27 }
28
29 @Override
30 public String toString() {
31     return "Student{" +
32         "name='" + name + '\'' +
33         ", age=" + age +
34         "'}";
35 }
36 }
37
38 public class TestDemo {
39     public static void main(String[] args) {
40         /*
41         1.通过getClass获取Class对象
42         */
43         Student s1 = new Student();
44         Class c1 = s1.getClass();
45         /*
46         2.直接通过 类名.class 的方式得到,该方法最为安全可靠,程序性能更高
47         这说明任何一个类都有一个隐含的静态成员变量 class
48         */
49         Class c2 = Student.class;
50         /*
51         3.通过 Class 对象的 forName() 静态方法来获取,用的最多,
52         但可能抛出 ClassNotFoundException 异常
```

```

53         */
54         Class c3 = null;
55         try {
56             //注意这里是类的全路径，如果有包需要加包的路径
57             c3 = Class.forName("Student");
58         } catch (ClassNotFoundException e) {
59             e.printStackTrace();
60         }
61         //一个类在 JVM 中只会有一个 Class 实例,即我们对上面获取的
62         //c1,c2,c3进行 equals 比较,发现都是true
63         System.out.println(c1.equals(c2));
64         System.out.println(c1.equals(c3));
65         System.out.println(c2.equals(c3));
66     }
67 }

```

### 1.3.2.2 反射的使用

接下来我们开始使用反射，我们依旧反射上面的Student类，把反射的逻辑写到另外的类当中进行理解

**注意：**所有和反射相关的包都在 `import java.lang.reflect` 包下面。

```

1  import java.lang.reflect.Constructor;
2  import java.lang.reflect.Field;
3  import java.lang.reflect.Method;
4
5  public class ReflectClassDemo {
6      // 创建对象
7      public static void reflectNewInstance() {
8          try {
9              Class<?> classStudent = Class.forName("Student");
10             Object objectStudent = classStudent.newInstance();
11             Student student = (Student) objectStudent;
12             System.out.println("获得学生对象: "+student);
13         } catch (Exception ex) {
14             ex.printStackTrace();
15         }
16     }
17     // 反射私有的构造方法 屏蔽内容为获得公有的构造方法
18     public static void reflectPrivateConstructor() {
19         try {
20             Class<?> classStudent = Class.forName("Student");
21             //注意传入对应的参数
22             Constructor<?> declaredConstructorStudent =
classStudent.getDeclaredConstructor(String.class,int.class);

```

```

23         //Constructor<?> declaredConstructorStudent =
    classStudent.getConstructor();
24         //设置为true后可修改访问权限
25         declaredConstructorStudent.setAccessible(true);
26         Object objectStudent = declaredConstructorStudent.newInstance("高
    博",15);
27         //Object objectStudent = declaredConstructorStudent.newInstance();
28         Student student = (Student) objectStudent;
29         System.out.println("获得私有构造哈数且修改姓名和年龄: "+student);
30     } catch (Exception ex) {
31         ex.printStackTrace();
32     }
33 }
34
35 // 反射私有属性
36 public static void reflectPrivateField() {
37     try {
38         Class<?> classStudent = Class.forName("Student");
39
40         Field field = classStudent.getDeclaredField("name");
41         field.setAccessible(true);
42         //可以修改该属性的值
43         Object objectStudent = classStudent.newInstance();
44         Student student = (Student) objectStudent;
45
46         field.set(student,"小明");
47
48         String name = (String) field.get(student);
49         System.out.println("反射私有属性修改了name: "+ name);
50     } catch (Exception ex) {
51         ex.printStackTrace();
52     }
53 }
54
55 // 反射私有方法
56 public static void reflectPrivateMethod() {
57     try {
58         Class<?> classStudent = Class.forName("Student");
59         Method methodStudent =
    classStudent.getDeclaredMethod("function",String.class);
60         System.out.println("私有方法的方法名为: "+methodStudent.getName());
61         //私有的一般都要加
62         methodStudent.setAccessible(true);
63         Object objectStudent = classStudent.newInstance();
64         Student student = (Student) objectStudent;
65         methodStudent.invoke(student,"我是给私有的function函数传的参数");
66

```

```
67         } catch (Exception ex) {
68             ex.printStackTrace();
69         }
70     }
71
72     public static void main(String[] args) {
73         //reflectNewInstance();
74         //reflectPrivateConstructor();
75         //reflectPrivateField();
76         reflectPrivateMethod();
77     }
78 }
```

## 1.4 反射优点和缺点

### 优点：

1. 对于任意一个类，都能够知道这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意一个方法
2. 增加程序的灵活性和扩展性，降低耦合性，提高自适应能力
3. 反射已经运用在了很多流行框架如：Struts、Hibernate、Spring 等等。

### 缺点：

1. 使用反射会有效率问题。会导致程序效率降低。具体参考这里：  
<http://www.imooc.com/article/293679>
2. 反射技术绕过了源代码的技术，因而会带来维护问题。反射代码比相应的直接代码更复杂。

## 1.5 重点总结

1. 反射的意义
2. 反射重要的几个类：Class类、Field类、Method类、Constructor类
3. 学会合理利用反射，一定要在安全环境下使用。

## 2. 枚举的使用

### 2.1 背景及定义

枚举是在JDK1.5以后引入的。主要用途是：将一组常量组织起来，在这之前表示一组常量通常使用定义常量的方式：

```
1 public static final int RED = 1;
2 public static final int GREEN = 2;
3 public static final int BLACK = 3;
```

但是常量举例有不好的地方，例如：可能碰巧有个数字1，但是他有可能误会为是RED，现在我们可以直接用枚举来进行组织，这样一来，就拥有了类型，枚举类型。而不是普通的整形1。

```
1 public enum TestEnum {
2     RED,BLACK,GREEN;
3 }
```

优点：将常量组织起来统一进行管理

场景：错误状态码，消息类型，颜色的划分，状态机等等....

本质：是 `java.lang.Enum` 的子类，也就是说，自己写的枚举类，就算没有显示的继承 `Enum`，但是其默认继承了这个类。

## 2.2 使用

### 1. switch语句

```
1 public enum TestEnum {
2     RED,BLACK,GREEN,WHITE;
3     public static void main(String[] args) {
4         TestEnum testEnum2 = TestEnum.BLACK;
5         switch (testEnum2) {
6             case RED:
7                 System.out.println("red");
8                 break;
9             case BLACK:
10                System.out.println("black");
11                break;
12             case WHITE:
13                System.out.println("WHITE");
14                break;
15             case GREEN:
16                System.out.println("black");
17                break;
18             default:
19                break;
20         }
21     }
22 }
```

### 2. 常用方法



| 方法名称        | 描述               |
|-------------|------------------|
| values()    | 以数组形式返回枚举类型的所有成员 |
| ordinal()   | 获取枚举成员的索引位置      |
| valueOf()   | 将普通字符串转换为枚举实例    |
| compareTo() | 比较两个枚举成员在定义时的顺序  |

示例一：

```
1 public enum TestEnum {  
2     RED, BLACK, GREEN, WHITE;  
3     public static void main(String[] args) {  
4         TestEnum[] testEnum2 = TestEnum.values();  
5         for (int i = 0; i < testEnum2.length; i++) {  
6             System.out.println(testEnum2[i] + " " + testEnum2[i].ordinal());  
7         }  
8         System.out.println("-----");  
9         System.out.println(TestEnum.valueOf("GREEN"));  
10    }  
11 }
```

示例二：

```
1 public enum TestEnum {  
2     RED, BLACK, GREEN, WHITE;  
3     public static void main(String[] args) {  
4         //拿到枚举实例BLACK  
5         TestEnum testEnum = TestEnum.BLACK;  
6         //拿到枚举实例RED  
7         TestEnum testEnum21 = TestEnum.RED;  
8         System.out.println(testEnum.compareTo(testEnum21));  
9         System.out.println(BLACK.compareTo(RED));  
10        System.out.println(RED.compareTo(BLACK));  
11    }  
12 }
```

刚刚说过，在Java当中枚举实际上就是一个类。所以我们在定义枚举的时候，还可以这样定义和使用枚举：

**重要：枚举的构造方法默认是私有的**

```

1 public enum TestEnum {
2     RED("red",1),BLACK("black",2),WHITE("white",3),GREEN("green",4);
3     private String name;
4     private int key;
5
6     /**
7      * 1、当枚举对象有参数后，需要提供相应的构造函数
8      * 2、枚举的构造函数默认是私有的 这个一定要记住
9      * @param name
10     * @param key
11     */
12     private TestEnum (String name,int key) {
13         this.name = name;
14         this.key = key;
15     }
16
17     public static TestEnum getEnumKey (int key) {
18         for (TestEnum t: TestEnum.values()) {
19             if(t.key == key) {
20                 return t;
21             }
22         }
23         return null;
24     }
25
26     public static void main(String[] args) {
27         System.out.println(getEnumKey(2));
28     }
29
30 }

```

## 2.3 枚举优点缺点

优点：

1. 枚举常量更简单安全。
2. 枚举具有内置方法，代码更优雅

缺点：

1. 不可继承，无法扩展

## 2.4 枚举和反射

### 2.4.1 枚举是否可以通过反射，拿到实例对象呢？

我们刚刚在反射里边看到了，任何一个类，哪怕其构造方法是私有的，我们也可以通过反射拿到他的实例对象，那么枚举的构造方法也是私有的，我们是否可以拿到呢？接下来，我们来实验一下：

同样利用上述提供的枚举类来进行举例：

```
1 public enum TestEnum {
2
3     RED("red",1),BLACK("black",2),WHITE("white",3),GREEN("green",4);
4     private String name;
5     private int key;
6
7     /**
8      * 1、当枚举对象有参数后，需要提供相应的构造函数
9      * 2、枚举的构造函数默认是私有的 这个一定要记住
10     * @param name
11     * @param key
12     */
13     private TestEnum (String name,int key) {
14         this.name = name;
15         this.key = key;
16     }
17
18     public static TestEnum getEnumKey (int key) {
19         for (TestEnum t: TestEnum.values()) {
20             if(t.key == key) {
21                 return t;
22             }
23         }
24         return null;
25     }
26
27     public static void reflectPrivateConstructor() {
28         try {
29             Class<?> classStudent = Class.forName("TestEnum");
30             //注意传入对应的参数,获得对应的构造方法来构造对象,当前枚举类是提供了两个参数
31             //分别是String和int。
32             Constructor<?> declaredConstructorStudent =
33             classStudent.getDeclaredConstructor(String.class,int.class);
34             //设置为true后可修改访问权限
35             declaredConstructorStudent.setAccessible(true);
36             Object objectStudent = declaredConstructorStudent.newInstance("绿
37             色",666);
38             TestEnum testEnum = (TestEnum) objectStudent;
39             System.out.println("获得枚举的私有构造函数: "+testEnum);
40         } catch (Exception ex) {
41             ex.printStackTrace();
42         }
43     }
44 }
```

```

39     }
40 }
41
42 public static void main(String[] args) {
43     reflectPrivateConstructor();
44 }
45
46 }

```

输出结果：

```

1 java.lang.NoSuchMethodException: TestEnum.<init>(java.lang.String, int)
2     at java.lang.Class.getConstructor0(Class.java:3082)
3     at java.lang.Class.getDeclaredConstructor(Class.java:2178)
4     at TestEnum.reflectPrivateConstructor(TestEnum.java:40)
5     at TestEnum.main(TestEnum.java:54)

```

我们提供的枚举的构造方法就是两个参数分别是 `String` 和 `int`，这里的异常信息是

`java.lang.NoSuchMethodException: TestEnum.<init>(java.lang.String, int)` 枚举比较特殊，虽然我们写的是两个，但是默认他还添加了两个参数，哪两个参数呢？我们看一下Enum类的源码：

```

1 protected Enum(String name, int ordinal) {
2     this.name = name;
3     this.ordinal = ordinal;
4 }

```

也就是说，我们自己的构造函数有两个参数一个是String一个是int，同时他默认还会给两个参数，一个是String一个是int。也就是说，这里我们正确给的是4个参数：

```

1 public static void reflectPrivateConstructor() {
2     try {
3         Class<?> classStudent = Class.forName("TestEnum");
4         //注意传入对应的参数,获得对应的构造方法来构造对象,当前枚举类是提供了两个参数分别是String和int。
5         Constructor<?> declaredConstructorStudent =
6             classStudent.getDeclaredConstructor(String.class, int.class, String.class, int.class);
7         //设置为true后可修改访问权限
8         declaredConstructorStudent.setAccessible(true);
9     } catch (Exception e) {
10        e.printStackTrace();
11    }
12 }

```

```
8      //后两个为子类参数，大家可以将当前枚举类的key类型改为double验证
9      Object objectStudent = declaredConstructorStudent.newInstance("父类参
    数",666,"子类参数",888);
10     TestEnum testEnum = (TestEnum) objectStudent;
11     System.out.println("获得枚举的私有构造函数: "+testEnum);
12 } catch (Exception ex) {
13     ex.printStackTrace();
14 }
15 }
```

此时运行程序结果是：

```
1 java.lang.IllegalArgumentException: Cannot reflectively create enum objects
2     at java.lang.reflect.Constructor.newInstance(Constructor.java:416)
3     at TestEnum.reflectPrivateConstructor(TestEnum.java:46)
4     at TestEnum.main(TestEnum.java:55)
```

这2个参数对应是前两个，还是后两个？

这次就是我要的结果！此时的异常信息显示，是我的一个方法这个方法：`newInstance()` 报错了！没错，问题就是这里，我们来看一下这个方法的源码，为什么会抛出 `java.lang.IllegalArgumentException`：异常呢？

源码显示

```

@NotNull @CallerSensitive
public T newInstance(Object ... initargs)
    throws InstantiationException, IllegalAccessException,
           IllegalArgumentException, InvocationTargetException
{
    if (!override) {
        if (!Reflection.quickCheckMemberAccess(clazz, modifiers)) {
            Class<?> caller = Reflection.getCallerClass();
            checkAccess(caller, clazz, obj: null, modifiers);
        }
    }
    if ((clazz.getModifiers() & Modifier.ENUM) != 0)
        throw new IllegalArgumentException("Cannot reflectively create enum objects");
    ConstructorAccessor ca = constructorAccessor; // read volatile
    if (ca == null) {
        ca = acquireConstructorAccessor();
    }
    /unchecked/
    T inst = (T) ca.newInstance(initargs);
    return inst;
}

```

是的，枚举在这里被过滤了，你不能通过反射获取枚举类的实例！这道题是2017年阿里巴巴曾经问到的一个问题，不看不知道，一看吓一跳！同学们记住这个坑。**原版问题是：为什么枚举实现单例模式是安全的？**希望同学们记住这个问题！

## 2.5 总结

- 1、枚举本身就是一个类，其构造方法默认为私有的，且都是默认继承与 `java.lang.Enum`
- 2、枚举可以避免反射和序列化问题
- 3、枚举的优点和缺点

## 2.6 面试问题(单例模式学完后可以回顾):

1. 写一个单例模式。

```

1 public class Singleton {
2     private volatile static Singleton uniqueInstance;
3     private Singleton() {}
4     public static Singleton getInstance() {
5         if (uniqueInstance == null) {
6             synchronized (Singleton.class){

```

```

7         if(uniqueInstance == null){//进入区域后，再检查一次，如果仍是null,才
    创建实例
8             uniqueInstance = new Singleton();
9         }
10    }
11    }
12    return uniqueInstance;
13 }
14 }

```

## 2. 用静态内部类实现一个单例模式

```

1 class Singleton {
2     /** 私有化构造器 */
3     private Singleton() {
4
5     }
6     /** 对外提供公共的访问方法 */
7     public static Singleton getInstance() {
8         return UserSingletonHolder.INSTANCE;
9     }
10
11    /** 写一个静态内部类，里面实例化外部类 */
12    private static class UserSingletonHolder {
13        private static final Singleton INSTANCE = new Singleton();
14    }
15 }
16 public class Main {
17     public static void main(String[] args) {
18         Singleton u1 = Singleton.getInstance();
19         Singleton u2 = Singleton.getInstance();
20         System.out.println("两个实例是否相同: " + (u1==u2));
21     }
22 }

```

## 3. 用枚举实现一个单例模式

```

1 public enum TestEnum {
2     INSTANCE;
3     public TestEnum getInstance(){
4         return INSTANCE;
5     }
6     public static void main(String[] args) {

```

```
7      TestEnum singleton1=TestEnum.INSTANCE;
8      TestEnum singleton2=TestEnum.INSTANCE;
9      System.out.println("两个实例是否相同: "+(singleton1==singleton2));
10     }
11 }
```

## 3. Lambda表达式

### 3.1 背景

Lambda表达式是Java SE 8中一个重要的新特性。lambda表达式允许你通过表达式来代替功能接口。lambda表达式就和方法一样,它提供了一个正常的参数列表和一个使用这些参数的主体(body,可以是一个表达式或一个代码块)。**Lambda 表达式 (Lambda expression)** , 基于数学中的λ演算得名, 也可称为闭包 (Closure) 。

#### 3.1.1 Lambda表达式的语法

基本语法: **(parameters) -> expression** 或 **(parameters) -> { statements; }**

Lambda表达式由三部分组成:

1. **paramaters**: 类似方法中的形参列表, 这里的参数是函数式接口里的参数。这里的参数类型可以明确的声明也可不声明而由JVM隐含的推断。另外当只有一个推断类型时可以省略掉圆括号。
2. **->**: 可理解为“被用于”的意思
3. **方法体**: 可以是表达式也可以代码块, 是函数式接口里方法的实现。代码块可返回一个值或者什么都不返回, 这里的代码块块等同于方法的方法体。如果是表达式, 也可以返回一个值或者什么都不返回。

```
1 // 1. 不需要参数,返回值为 2
2 () -> 2
3
4 // 2. 接收一个参数(数字类型),返回其2倍的值
5 x -> 2 * x
6
7 // 3. 接受2个参数(数字),并返回他们的和
8 (x, y) -> x + y
9
10 // 4. 接收2个int型整数,返回他们的乘积
11 (int x, int y) -> x * y
12
13 // 5. 接受一个 string 对象,并在控制台打印,不返回任何值(看起来像是返回void)
14 (String s) -> System.out.print(s)
```



### 3.1.2 函数式接口

要了解Lambda表达式,首先需要了解什么是函数式接口,函数式接口定义:一个接口有且只有一个抽象方法。

注意:

1. 如果一个接口只有一个抽象方法,那么该接口就是一个函数式接口
2. 如果我们在某个接口上声明了 `@FunctionalInterface` 注解,那么编译器就会按照函数式接口的定义来要求该接口,这样如果有两个抽象方法,程序编译就会报错的。所以,从某种意义上来说,只要你保证你的接口中只有一个抽象方法,你可以不加这个注解。加上就会自动进行检测的。

定义方式:

```
1 @FunctionalInterface
2 interface NoParameterNoReturn {
3     //注意:只能有一个方法
4     void test();
5 }
```

但是这种方式也是可以的:

```
1 @FunctionalInterface
2 interface NoParameterNoReturn {
3     void test();
4     default void test2() {
5         System.out.println("JDK1.8新特性, default默认方法可以有具体的实现");
6     }
7 }
```

## 3.2 Lambda表达式的基本使用

首先,我们事先准备好几个接口:

```
1 //无返回值无参数
2 @FunctionalInterface
3 interface NoParameterNoReturn {
4     void test();
5 }
6 //无返回值一个参数
7 @FunctionalInterface
8 interface OneParameterNoReturn {
```

```

9     void test(int a);
10 }
11 //无返回值多个参数
12 @FunctionalInterface
13 interface MoreParameterNoReturn {
14     void test(int a,int b);
15 }
16 //有返回值无参数
17 @FunctionalInterface
18 interface NoParameterReturn {
19     int test();
20 }
21
22 //有返回值一个参数
23 @FunctionalInterface
24 interface OneParameterReturn {
25     int test(int a);
26 }
27 //有返回值多参数
28 @FunctionalInterface
29 interface MoreParameterReturn {
30     int test(int a,int b);
31 }

```

我们在上面提到过，Lambda可以理解为：Lambda就是匿名内部类的简化，实际上是创建了一个类，实现了接口，重写了接口的方法。

没有使用lambda表达式的时候的调用方式：

```

1 NoParameterNoReturn noParameterNoReturn = new NoParameterNoReturn(){
2     @Override
3     public void test() {
4         System.out.println("hello");
5     }
6 };
7
8 noParameterNoReturn.test();

```

具体使用见以下示例代码：

```

1 public class TestDemo {
2     public static void main(String[] args) {
3         NoParameterNoReturn noParameterNoReturn = ()->{

```

```

4         System.out.println("无参数无返回值");
5     };
6     noParameterNoReturn.test();
7     OneParameterNoReturn oneParameterNoReturn = (int a)->{
8         System.out.println("一个参数无返回值: "+ a);
9     };
10    oneParameterNoReturn.test(10);
11
12    MoreParameterNoReturn moreParameterNoReturn = (int a,int b)->{
13        System.out.println("多个参数无返回值: "+a+" "+b);
14    };
15    moreParameterNoReturn.test(20,30);
16
17    NoParameterReturn noParameterReturn = ()->{
18        System.out.println("有返回值无参数! ");
19        return 40;
20    };
21    //接收函数的返回值
22    int ret = noParameterReturn.test();
23    System.out.println(ret);
24    OneParameterReturn oneParameterReturn = (int a)->{
25        System.out.println("有返回值有一个参数! ");
26        return a;
27    };
28
29    ret = oneParameterReturn.test(50);
30    System.out.println(ret);
31
32    MoreParameterReturn moreParameterReturn = (int a,int b)->{
33        System.out.println("有返回值多个参数! ");
34        return a+b;
35    };
36    ret = moreParameterReturn.test(60,70);
37    System.out.println(ret);
38 }
39 }

```

### 3.2.1 语法精简

1. 参数类型可以省略，如果需要省略，每个参数的类型都要省略。
2. 参数的小括号里面只有一个参数，那么小括号可以省略
3. 如果方法体当中只有一句代码，那么大括号可以省略
4. 如果方法体中只有一条语句，且是return语句，那么大括号可以省略，且去掉return关键字。

示例代码：

```

1 public static void main(String[] args) {
2     MoreParameterNoReturn moreParameterNoReturn = ( a, b)->{
3         System.out.println("无返回值多个参数, 省略参数类型: "+a+" "+b);
4     };
5     moreParameterNoReturn.test(20,30);
6
7     OneParameterNoReturn oneParameterNoReturn = a ->{
8         System.out.println("无参数一个返回值,小括号可以胜率: "+ a);
9     };
10    oneParameterNoReturn.test(10);
11
12    NoParameterNoReturn noParameterNoReturn = ()->System.out.println("无参数无返回值, 方法体中只有一行代码");
13    noParameterNoReturn.test();
14
15    //方法体中只有一条语句, 且是return语句
16    NoParameterReturn noParameterReturn = ()-> 40;
17    int ret = noParameterReturn.test();
18    System.out.println(ret);
19
20 }

```

### 3.3 变量捕获

Lambda 表达式中存在变量捕获，了解了变量捕获之后，我们才能更好的理解Lambda 表达式的作用域。Java当中的匿名类中，会存在变量捕获。

#### 3.3.1 3.1 匿名内部类

匿名内部类就是没有名字的内部类。我们这里只是为了说明变量捕获，所以，匿名内部类只要会使用就好，那么下面我们来，简单的看看匿名内部类的使用就好了。

具体想详细了解的同学戳这里：<https://www.cnblogs.com/SQP51312/p/6100314.html>

我们通过简单的代码来学习一下：

```

1 class Test {
2     public void func(){
3         System.out.println("func()");
4     }
5 }
6 public class TestDemo {
7     public static void main(String[] args) {
8         new Test(){
9             @Override

```

```

10         public void func() {
11             System.out.println("我是内部类，且重写了func这个方法！");
12         }
13     };
14 }
15 }

```

在上述代码当中的main函数当中，我们看到的就是一个匿名内部类的简单的使用。

### 3.3.2 匿名内部类的变量捕获

```

1 class Test {
2     public void func(){
3         System.out.println("func()");
4     }
5 }
6 public class TestDemo {
7     public static void main(String[] args) {
8         int a = 100;
9         new Test(){
10             @Override
11             public void func() {
12                 System.out.println("我是内部类，且重写了func这个方法！");
13                 System.out.println("我是捕获到变量 a == "+a
14                     +" 我是一个常量，或者是一个没有改变过值的变量！");
15             }
16         };
17     }
18 }

```

在上述代码当中的变量a就是，捕获的变量。这个变量要么是被final修饰，如果不是被final修饰的 你要保证在使用之前，没有修改。如下代码就是错误的代码。

```

1 public class TestDemo {
2     public static void main(String[] args) {
3         int a = 100;
4         new Test(){
5             @Override
6             public void func() {
7                 a = 99;
8                 System.out.println("我是内部类，且重写了func这个方法！");
9                 System.out.println("我是捕获到变量 a == "+a
10                     +" 我是一个常量，或者是一个没有改变过值的变量！");

```

```

11         }
12     };
13 }
14 }

```

该代码直接编译报错。

### 3.3.3 Lambda的变量捕获

在Lambda当中也可以进行变量的捕获，具体我们看一下代码。

```

1 @FunctionalInterface
2 interface NoParameterNoReturn {
3     void test();
4 }
5
6 public static void main(String[] args) {
7     int a = 10;
8     NoParameterNoReturn noParameterNoReturn = ()->{
9         // a = 99; error
10        System.out.println("捕获变量: "+a);
11    };
12    noParameterNoReturn.test();
13 }

```

## 3.4 Lambda在集合当中的使用

为了能够让Lambda和Java的集合类集更好的一起使用，集合当中，也新增了部分接口，以便与Lambda表达式对接。

| 对应的接口      | 新增的方法                                                                                                                               |
|------------|-------------------------------------------------------------------------------------------------------------------------------------|
| Collection | removeIf() spliterator() stream() parallelStream()<br>forEach()                                                                     |
| List       | replaceAll() sort()                                                                                                                 |
| Map        | getOrDefault() forEach() replaceAll() putIfAbsent()<br>remove() replace() computeIfAbsent()<br>computeIfPresent() compute() merge() |

以上方法的作用可自行查看我们发的帮助手册。我们这里会示例一些方法的使用。注意：Collection的forEach()方法是从接口 `java.lang.Iterable` 拿过来的。

### 3.4.1 Collection接口

## forEach() 方法演示

该方法在接口 `Iterable` 当中，原型如下：

```
1 default void forEach(Consumer<? super T> action) {
2     Objects.requireNonNull(action);
3     for (T t : this) {
4         action.accept(t);
5     }
6 }
```

该方法表示：对容器中的每个元素执行action指定的动作。

```
1 public static void main(String[] args) {
2     ArrayList<String> list = new ArrayList<>();
3     list.add("Hello");
4     list.add("bit");
5     list.add("hello");
6     list.add("lambda");
7     list.forEach(new Consumer<String>(){
8         @Override
9         public void accept(String str){
10             //简单遍历集合中的元素。
11             System.out.print(str+" ");
12         }
13     });
14 }
```

输出结果：Hello bit hello lambda

我们可以修改为如下代码：

```
1 public static void main(String[] args) {
2     ArrayList<String> list = new ArrayList<>();
3     list.add("Hello");
4     list.add("bit");
5     list.add("hello");
6     list.add("lambda");
7     //表示调用一个，不带有参数的方法，其执行花括号内的语句，为原来的函数体内容。
8     list.forEach(s -> {
9         System.out.println(s);
10     });
11 }
```

输出结果：Hello bit hello lambda

### 3.4.2 List接口

#### sort()方法的演示

sort方法源码：该方法根据c指定的比较规则对容器元素进行排序。

```
1 public void sort(Comparator<? super E> c) {
2     final int expectedModCount = modCount;
3     Arrays.sort((E[]) elementData, 0, size, c);
4     if (modCount != expectedModCount) {
5         throw new ConcurrentModificationException();
6     }
7     modCount++;
8 }
```

使用示例：

```
1 public static void main(String[] args) {
2     ArrayList<String> list = new ArrayList<>();
3     list.add("Hello");
4     list.add("bit");
5     list.add("hello");
6     list.add("lambda");
7     list.sort(new Comparator<String>() {
8         @Override
9         public int compare(String str1, String str2){
10             //注意这里比较长度
11             return str1.length()-str2.length();
12         }
13     });
14     System.out.println(list);
15 }
```

输出结果：bit, Hello, hello, lambda

修改为lambda表达式：

```
1 public static void main(String[] args) {
2     ArrayList<String> list = new ArrayList<>();
3     list.add("Hello");
```



```

4    list.add("bit");
5    list.add("hello");
6    list.add("lambda");
7    //调用带有2个参数的方法，且返回长度的差值
8    list.sort((str1,str2)-> str1.length()-str2.length());
9    System.out.println(list);
10 }

```

输出结果：bit, Hello, hello, lambda

### 3.4.3 Map接口

#### HashMap 的 forEach()

该方法原型如下：

```

1  default void forEach(BiConsumer<? super K, ? super V> action) {
2      Objects.requireNonNull(action);
3      for (Map.Entry<K, V> entry : entrySet()) {
4          K k;
5          V v;
6          try {
7              k = entry.getKey();
8              v = entry.getValue();
9          } catch (IllegalStateException ise) {
10             // this usually means the entry is no longer in the map.
11             throw new ConcurrentModificationException(ise);
12          }
13          action.accept(k, v);
14      }
15 }

```

作用是对Map中的每个映射执行action指定的操作。

代码示例：

```

1  public static void main(String[] args) {
2      HashMap<Integer, String> map = new HashMap<>();
3      map.put(1, "hello");
4      map.put(2, "bit");
5      map.put(3, "hello");
6      map.put(4, "lambda");
7      map.forEach(new BiConsumer<Integer, String>(){
8          @Override
9          public void accept(Integer k, String v){

```

```
10         System.out.println(k + "=" + v);
11     }
12 });
13 }
```

输出结果：

1=hello

2=bit

3=hello

4=lambda

使用lambda表达式后的代码：

```
1 public static void main(String[] args) {
2     HashMap<Integer, String> map = new HashMap<>();
3     map.put(1, "hello");
4     map.put(2, "bit");
5     map.put(3, "hello");
6     map.put(4, "lambda");
7     map.forEach((k,v)-> System.out.println(k + "=" + v));
8 }
```

输出结果：

1=hello

2=bit

3=hello

4=lambda

### 3.5 总结

Lambda表达式的优点很明显，在代码层次上来说，使代码变得非常的简洁。缺点也很明显，代码不易读。

优点：

1. 代码简洁，开发迅速
2. 方便函数式编程
3. 非常容易进行并行计算
4. Java 引入 Lambda，改善了集合操作

### 缺点：

1. 代码可读性变差
  2. 在非并行计算中，很多计算未必有传统的 for 性能要高
  3. 不容易进行调试
- 

完

比特就业课